

Laboratorium 9: Organizacja pamięci, adresacja i wskaźniki

Opracowanie zestawu zadań (część programistyczna)

Kacper Bednarczuk

Kod programu

Program został napisany w języku C, co pozwala na bezpośrednią manipulację pamięcią i wskaźnikami. Kod realizuje trzy eksperymenty: weryfikację kolejności bajtów (Endianness), badanie kierunku wzrostu stosu oraz analizę wyrównania struktur (padding).

```
#include <stdio.h>

struct Student {
    char initial;        // 1 bajt
    // Padding: 3 bajty (wyrównanie inta do adresu podzielnego przez 4)
    int studentId;       // 4 bajty
    short age;           // 2 bajty
    // Padding: 2 bajty (wyrównanie całej struktury do wielokrotności 4)
};

struct StudentOptimised {
    char initial;        // 1 bajt
    // Padding: 1 bajt (wyrównanie shorta do adresu podzielnego przez 2)
    short age;           // 2 bajty
    int studentId;       // 4 bajty
    // Brak paddingu na końcu (rozmiar 8 jest wielokrotnością 4)
};

void checkEndianness() {
    printf("=== Zadanie 2.1: Endianness ===\n");
    unsigned int value = 0x11223344;
    // Rzutowanie wskaźnika, aby dostać się do pojedynczych bajtów
    unsigned char *ptr = (unsigned char*)&value;

    printf("Wartość hex: 0x%X\n", value);
    for (int i = 0; i < 4; i++) {
        printf("Bajt %d: 0x%02X\n", i, ptr[i]);
    }

    // Sprawdzenie pierwszego bajtu (pod najniższym adresem)
```

```

    if (ptr[0] == 0x44) {
        printf("Wniosek: Twój system to Little-Endian\n");
    } else if (ptr[0] == 0x11) {
        printf("Wniosek: Twój system to Big-Endian\n");
    } else {
        printf("Wniosek: Inny/Nieznany układ\n");
    }
    printf("\n");
}

void checkAllocationDirection() {
    printf("=== Zadanie 2.2: Kierunek alokacji na stosie ===\n");
    int variableA;
    int variableB;

    printf("Adres zmiennej A: %p\n", (void*)&variableA);
    printf("Adres zmiennej B: %p\n", (void*)&variableB);

    // Porównanie adresów
    if (&variableA > &variableB) {
        printf("Wniosek: Nowe zmienne otrzymują NIŻSZE adresy. (Alokacja postępuje w stronę adresu 0).\n");
    } else {
        printf("Wniosek: Nowe zmienne otrzymują WYŻSZE adresy. (Alokacja postępuje w stronę maksymalnego adresu).\n");
    }
    printf("\n");
}

void checkStructPadding() {
    printf("=== Zadanie 2.3: Struktury i Padding ===\n");

    printf("Rozmiar struct Student: %zu bajtów\n", sizeof(struct Student));
    printf("Rozmiar struct StudentOptimised: %zu bajtów\n", sizeof(struct StudentOptimised));

    size_t savedBytes = sizeof(struct Student) - sizeof(struct StudentOptimised);
    printf("Zaoszczędzono: %zu bajtów\n", savedBytes);
    printf("\n");
}

int main() {
    checkEndianness();
    checkAllocationDirection();
    checkStructPadding();

    return 0;
}

```

Wyniki działania programu

Poniżej przedstawiono logi wygenerowane przez program uruchomiony w konsoli systemowej:

```
=== Zadanie 2.1: Endianness ===
Wartość hex: 0x11223344
Bajt 0: 0x44
Bajt 1: 0x33
Bajt 2: 0x22
Bajt 3: 0x11
Wniosek: Twój system to Little-Endian

=== Zadanie 2.2: Kierunek alokacji na stosie ===
Adres zmiennej A: 0x7ffc3fd8a0ac
Adres zmiennej B: 0x7ffc3fd8a0a8
Wniosek: Nowe zmienne otrzymują NIŻSZE adresy. (Alokacja postępuje w stronę adresu 0).

=== Zadanie 2.3: Struktury i Padding ===
Rozmiar struct Student: 12 bajtów
Rozmiar struct StudentOptimised: 8 bajtów
Zaoszczędzono: 4 bajtów
```

Analiza i wnioski

1. Endianness (Kolejność bajtów)

Eksperyment potwierdził, że testowana maszyna pracuje w trybie **Little-Endian**. Dla liczby `0x11223344`, bajt najmniej znaczący (`0x44`) znalazł się pod najniższym adresem w pamięci (Bajt 0), co jest charakterystyczne dla architektury procesorów x86/x64 (Intel/AMD).

2. Organizacja Stosu

Porównanie adresów dwóch kolejno zadeklarowanych zmiennych lokalnych (`variableA` i `variableB`) wykazało, że:

$$Adres_A(\dots0ac) > Adres_B(\dots0a8)$$

Różnica wynosi dokładnie 4 bajty ($C_{16} - 8_{16} = 4_{10}$), co odpowiada rozmiarowi typu `int`. Fakt, że druga zmienna otrzymała niższy adres, potwierdza, że na tym systemie kierunek alokacji stosu postępuje w stronę adresu 0.

3. Optymalizacja struktur (Padding)

Program wykazał różnicę w rozmiarze struktur przechowujących te same dane, ale w innej kolejności:

- `struct Student` (**12 bajtów**): Nieoptymalny układ wymusił wstawienie łącznie 5 bajtów paddingu (3 bajty wewnątrz + 2 bajty na końcu).
- `struct StudentOptimised` (**8 bajtów**): Przesunięcie pola `short` obok `char` zredukowało padding do zaledwie 1 bajta.

Dzięki prostej zmianie kolejności pól w kodzie, zaoszczędzono **4 bajty** pamięci na każdej instancji struktury, co stanowi zysk rzędu **33%**.